YUMI RAG + Next.js

Implementation, connection, and PowerShell verification guide

This guide follows the current portfolio repository. It documents the PDF ingestion and retrieval flow, the FastAPI and Next.js connection, and practical PowerShell commands to start and inspect the local services. Environment files and secret values are intentionally excluded. Replace development credentials with secure values before any public deployment.

1. How the RAG system works

- Ingestion: PyMuPDF extracts each PDF page, chunk_text splits page text into overlapping sections, Sentence Transformers embeds each section, and PostgreSQL stores text plus vector embeddings.
- Retrieval: the Python API embeds the question, searches pgvector by vector distance, and returns the closest relevant chunks with filename and page metadata.
- Generation: the RAG pipeline builds a grounded prompt from the question, recent chat history, and retrieved chunks. Ollama generates an answer; the backend attaches citation labels and source records.
- Connection: the browser sends the chat request to Next.js /api/chat. That server route forwards it to FastAPI /api/chat. FastAPI calls the RAG pipeline and returns JSON to the browser.

Request path: FloatingChat.tsx -> Next.js route.ts -> FastAPI app/api/chat.py -> rag/pipeline.py -> PostgreSQL/pgvector + Ollama.

2. Configure and start locally

Create or update the local Next.js environment file (.env.local) with the Python backend address. Do not paste secret values into this guide.

```
PYTHON_API_URL=http://localhost:8000
```

When Next.js itself runs inside Docker on the same Compose network, use the backend service name instead: PYTHON_API_URL=http://backend:8000. The current Compose file runs the backend services; start Next.js separately with pnpm dev.

Step-by-step: what each part does

Step 1 - Database schema

backend/schema.sql enables pgvector, creates pdf_history to track each source PDF, and creates pdf_chunk to hold page text and 384-number embeddings. Retrieval compares question vectors to these stored vectors.

Step 2 - Extract and split the PDF

backend/app/rag/pdf_extractor.py defines PdfPage and PdfResult and uses extract_pdf_text() to read text page by page. backend/app/rag/chunker.py defines chunk_text(); its 1500-character size and 200-character overlap keep related text together across boundaries.

Step 3 - Embed and ingest

backend/app/rag/embeddings.py defines get_model(), create_embedding(), and create_embeddings(). The cached SentenceTransformer maps text to normalized vectors. backend/app/rag/ingestion.py defines ingest_pdf(): it removes the prior record with the same filename, extracts each page, chunks it, embeds each chunk, and stores everything in one database transaction. backend/ingest.py selects the mounted portfolio PDF and starts ingestion.

Step 4 - Retrieve relevant text

backend/app/rag/retrieval.py defines retrieve_relevant_chunks(). It embeds the question, uses pgvector's cosine-distance operator to rank chunks, applies max_distance, and removes near-duplicate chunks. The filename and page are retained for citations.

Step 5 - Interpret the question

backend/app/rag/query_rewriter.py defines is_follow_up_question() and rewrite_query(). These combine a short follow-up with the previous user question. backend/app/rag/pipeline.py then applies portfolio-specific project and technology-stack search hints, handles greetings and thanks directly, and formats project/stack pairs from explicit document headings.

Step 6 - Build a grounded answer

backend/app/rag/prompt.py defines build_rag_prompt(). It gives Ollama only the question, recent history, and retrieved PDF text, and asks the model to cite a source. backend/app/rag/llm.py defines generate_text(), which calls Ollama's /api/chat endpoint with the configured model and generation limits.

Step 7 - Validate and return

answer_question() in pipeline.py runs the complete retrieval/generation flow, checks for a usable answer, adds a source label if needed, and returns answer plus source records. Deterministic formats are used where the PDF's project-to-stack mapping should not be left to a small model.

Step 8 - Expose FastAPI endpoints

backend/app/api/chat.py defines ChatMessage and ChatRequest and the POST /api/chat endpoint. backend/app/main.py creates the FastAPI app, registers the router, enables development CORS, and defines GET /health. backend/Dockerfile and backend/requirements.txt build and install this service.

Step 9 - Connect Next.js

src/app/api/chat/route.ts reads PYTHON_API_URL on the server, forwards JSON to FastAPI, and returns JSON errors for bad status, invalid JSON, or connection failures. src/components/FloatingChat.tsx owns the chat UI state; handleSubmit() posts to the same-origin Next.js route, appends user/assistant messages, and shows loading, errors, and sources.

Step 10 - Run the services

compose.yaml defines PostgreSQL/pgvector, Ollama, and FastAPI, their Docker DNS names, ports, document mount, and persistent model/data volumes. package.json supplies pnpm dev, pnpm lint, and pnpm build for the Next.js side.
The full source appendix follows these steps in dependency order. Function-level explanations are also included in the files' comments; use the filenames in each heading to open the copyable, unwrapped source in the repository.

PowerShell: start services

This starts the database, local model server, and Python API in the background. The second command confirms their current state.

```
docker compose up -d db ollama backend
docker compose ps
docker compose exec ollama ollama pull llama3.2:1b
docker compose exec ollama ollama list
```

Apply schema and ingest the PDF

The first command creates the pgvector extension and tables in the running database. The second extracts and stores the mounted PDF. Re-ingesting the same filename replaces its stored history and chunks.

```
Get-Content .\backend\schema.sql | docker compose exec -T db psql -U postgres -d yumi
docker compose exec backend python ingest.py
```

Run the Next.js app

Install the JavaScript packages once, then start the development server. Keep this terminal open while using the portfolio.

```
pnpm install
pnpm dev
```

3. PowerShell checks and request examples

Check service health and model availability

These commands verify the FastAPI health endpoint, Ollama API/model, Docker service status, and number of stored PDF chunks.

```
Invoke-RestMethod http://localhost:8000/health
Invoke-RestMethod http://localhost:11434/api/tags
docker compose ps
docker compose exec ollama ollama list
docker compose exec db psql -U postgres -d yumi -c "SELECT COUNT(*) AS chunk_count FROM pdf_chunk;"
```

Send a question directly to FastAPI

This creates a JSON request with the question and empty chat history, then calls the Python endpoint directly. A successful response contains answer and sources fields.

```
$body = @{ question = "List all projects and their tech stacks"; messages = @() } | ConvertTo-Json -Depth 6
Invoke-RestMethod -Method Post -Uri http://localhost:8000/api/chat -ContentType "application/json" -Body $body
```

Check the Next.js proxy

This checks that the browser-facing Next.js route can forward a request to Python and relay its JSON answer.

```
$body = @{ question = "What is EventSphere?"; messages = @() } | ConvertTo-Json -Depth 6
Invoke-RestMethod -Method Post -Uri http://localhost:3000/api/chat -ContentType "application/json" -Body $body
```

View logs and run available checks

Docker logs help locate Python, database, and Ollama errors. pnpm lint checks frontend code style and pnpm build compiles the production frontend.

```
docker compose logs --tail=100 backend ollama db
pnpm lint
pnpm build
```

The current package.json does not define a test script, and no dedicated test files are present in the repository. pnpm lint and pnpm build are the available frontend checks; the PowerShell HTTP examples exercise the running chat API.

Automated PowerShell integration smoke test

scripts/Test-YumiRag.ps1 checks FastAPI health, confirms the expected Ollama model is installed, asks the backend for all listed projects and tech stacks, and repeats the question through Next.js. It verifies the three project names, a tech-stack section, and returned PDF sources. It exits with code 1 and prints diagnostics when a check fails.

Verified in this workspace on 2026-10-23: all 4 checks passed (FastAPI health, installed Ollama model, direct backend RAG response, and Next.js proxy response).

Start the services and Next.js first, then run:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\Test-YumiRag.ps1
```

To point it at different local ports or another model, pass parameters:

```
.\scripts\Test-YumiRag.ps1 -BackendUrl http://localhost:8000 -NextUrl http://localhost:3000 -OllamaUrl
    http://localhost:11434 -ModelName llama3.2:1b
```

Common failure clues

- ECONNREFUSED localhost:8000: verify the backend is running and PYTHON_API_URL points to the right host from where Next.js is running.
- Information not found: verify the PDF was ingested and the relevant text exists in pdf_chunk; inspect the response and backend logs.
- Ollama timeout: confirm the model appears in ollama list and check Ollama logs. The first model load can be slow on CPU.
- Invalid JSON from backend: inspect docker compose logs --tail=100 backend; the API route now reports backend errors in Next.js logs.

4. Current project source code

The following appendix reproduces the implementation files from this workspace at the time this guide was generated. Long lines are wrapped for page width; use the repository files as the copyable source of truth.

backend/schema.sql

```
-- Enable pgvector for semantic search.
CREATE EXTENSION IF NOT EXISTS vector;

-- Store each ingested PDF and its extracted text.
CREATE TABLE IF NOT EXISTS public.pdf_history (
    id BIGSERIAL PRIMARY KEY,
    user_id TEXT,
    file_name TEXT NOT NULL,
    extracted_text TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Store page-level text chunks and their 384-dimensional embeddings.
CREATE TABLE IF NOT EXISTS public.pdf_chunk (
    id BIGSERIAL PRIMARY KEY,
    pdf_history_id BIGINT NOT NULL
        REFERENCES public.pdf_history(id) ON DELETE CASCADE,
    text TEXT NOT NULL,
    page INTEGER NOT NULL,
    chunk_index INTEGER NOT NULL,
    embedding VECTOR(384) NOT NULL
);

CREATE INDEX IF NOT EXISTS pdf_chunk_history_id_idx
    ON public.pdf_chunk (pdf_history_id);
```

backend/requirements.txt

```
fastapi
uvicorn[standard]
```

```
python-dotenv
pymupdf
sentence-transformers
psycopg[binary]
sqlalchemy
httpx
```

backend/Dockerfile

```
# Use Python 3.12 with the smaller slim image
FROM python:3.12-slim

# Set the working directory inside the container
WORKDIR /app

# Prevent Python from creating .pyc files
ENV PYTHONDONTWRITEBYTECODE=1

# Make Python output appear immediately in Docker logs
ENV PYTHONUNBUFFERED=1

# Copy the dependency file into the container
COPY requirements.txt .

# Install all Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy the FastAPI application into the container
COPY app ./app

# Copy the PDF ingestion script into the container
COPY ingest.py .

# Tell Docker that the application uses port 8000
EXPOSE 8000

# Start the FastAPI application
#
# app.main:app means:
# app/main.py -> app = FastAPI(...)
#
# 0.0.0.0 allows the application to be accessed
# from outside the Docker container.
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

compose.yaml

```
# docker-compose.yml
#
# This file defines all the Docker containers
# needed for the YUMI RAG system.
#
# We have 3 main services:
#
# 1. db       -> PostgreSQL + pgvector
# 2. ollama   -> Local AI model server
# 3. backend  -> FastAPI RAG backend


services:

  # ============================================================
  # 1. POSTGRESQL DATABASE + PGVECTOR
  # ============================================================

  db:

    # Use the PostgreSQL 17 image with pgvector installed.
    #
    # pgvector is needed because your RAG system stores
    # and searches embedding vectors.
    image: pgvector/pgvector:pg17

    # Database configuration.
    environment:
      # PostgreSQL username.
      POSTGRES_USER: postgres

      # PostgreSQL password.
      POSTGRES_PASSWORD: postgres

      # Database name.
      POSTGRES_DB: yumi

    # Connect Docker's PostgreSQL port to your computer.
    #
    # Computer:5432
    #     ?
    # Docker:5432
    ports:
      - "5432:5432"
```

```
    # Store PostgreSQL data in a Docker volume.
    #
    # This means your database data survives
    # even if the container is recreated.
    volumes:
      - yumi_pgdata:/var/lib/postgresql/data


# ============================================================
# 2. OLLAMA
# ============================================================

ollama:

    # Ollama provides the local AI model server.
    image: ollama/ollama:latest

    # Expose Ollama's API to your computer.
    #
    # Computer:11434
    #     ?
    # Docker:11434
    ports:
      - "11434:11434"

    # Store downloaded AI models in a Docker volume.
    #
    # Without this volume, downloaded models could be
    # lost when the container is recreated.
    volumes:
      - yumi_ollama:/root/.ollama


# ============================================================
# 3. FASTAPI BACKEND
# ============================================================

backend:

    # Build the backend image using the Dockerfile
    # located inside the ./backend folder.
    build:
      context: ./backend

    # Start these services before the backend.
    #
    # The backend needs:
    # - PostgreSQL
    # - Ollama
    depends_on:
      - db
      - ollama

    # Environment variables available inside
    # the backend container.
    environment:

      # IMPORTANT:
      #
      # Inside Docker, we DON'T use localhost for PostgreSQL.
      #
      # "db" is the service name above.
      #
      # Docker allows the backend to reach PostgreSQL using:
      #
      # db:5432
      DATABASE_URL: postgresql+psycopg://postgres:postgres@db:5432/yumi

      # IMPORTANT:
      #
      # Inside Docker, Ollama is reached using
      # the service name "ollama".
      #
      # Therefore:
      #
      # http://ollama:11434
      OLLAMA_HOST: http://ollama:11434

      # AI model that Ollama should use.
      OLLAMA_MODEL: llama3.2:1b

      # Persist the downloaded sentence-transformer model across rebuilds.
      HF_HOME: /root/.cache/huggingface

    # Expose FastAPI to your computer.
    #
    # Computer:8000
    #     ?
    # Docker:8000
    ports:
      - "8000:8000"
```

```
    # Make your local documents folder available
    # inside the backend container.
    #
    # Computer:
    # ./documents
    #
    # Docker:
    # /app/documents
    volumes:
      - ./documents:/app/documents
      - yumi_hf_cache:/root/.cache/huggingface


# ============================================================
# DOCKER VOLUMES
# ============================================================

# Docker-managed storage.
#
# These volumes keep important data even when
# containers are stopped or recreated.
volumes:

  # PostgreSQL database storage.
  yumi_pgdata:

  # Ollama model storage.
  yumi_ollama:

  # Hugging Face / Sentence Transformers model cache.
  yumi_hf_cache:
```

backend/ingest.py

```
from pathlib import Path

from app.rag.ingestion import ingest_pdf


if __name__ == "__main__":
    # Docker Compose mounts the project's documents folder at /app/documents.
    # Resolve the PDF relative to this script, independent of the current cwd.
    pdf_path = (
        Path(__file__).resolve().parent
        / "documents"
        / "INFORMATION OF KEMUEL PAUL.pdf"
    )
    count = ingest_pdf(str(pdf_path))
    print(f"Ingested {count} chunks from {pdf_path.name}.")
```

backend/app/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

# Import the chatbot router from app/api/chat.py.
#
# "router as chat_router" gives the imported router
# a clearer name in this file.
from app.api.chat import router as chat_router


# ------------------------------------------------------------
# CREATE THE FASTAPI APPLICATION
# ------------------------------------------------------------

# Create the main FastAPI application.
#
# The title appears in the automatic FastAPI documentation
# at:
#
# http://localhost:8000/docs
app = FastAPI(
    title="YUMI RAG API"
)


# ------------------------------------------------------------
# CORS CONFIGURATION
# ------------------------------------------------------------

# CORS allows your frontend and backend to communicate
# when they are running on different ports.
#
# Your setup is:
#
# Frontend:
# http://localhost:3000
#
# Backend:
```

```
# http://localhost:8000
#
# Because these are different origins, the backend needs
# to allow the frontend through CORS.
app.add_middleware(
    CORSMiddleware,

    # Only allow requests from your Next.js/React
    # development server.
    allow_origins=[
        "http://localhost:3000"
    ],

    # Allow cookies/authentication credentials
    # to be sent with requests.
    allow_credentials=True,

    # Allow all HTTP methods.
    #
    # Examples:
    # GET
    # POST
    # PUT
    # DELETE
    allow_methods=["*"],

    # Allow all request headers.
    allow_headers=["*"],
)


# ------------------------------------------------------------
# HEALTH CHECK
# ------------------------------------------------------------

# Create a simple endpoint that lets us check whether
# the FastAPI server is running.
@app.get("/health")
def health():

    # Return a simple JSON response.
    return {
        "status": "ok"
    }


# ------------------------------------------------------------
# REGISTER THE CHAT ROUTER
# ------------------------------------------------------------

# Add all routes from chat_router to the application.
#
# The prefix "/api" is added before the routes
# defined inside chat.py.
#
# chat.py contains:
#
# @router.post("/chat")
#
# Therefore the final endpoint becomes:
#
# POST /api/chat
app.include_router(
    chat_router,
    prefix="/api",
)
```

backend/app/api/chat.py

```
from fastapi import APIRouter
from pydantic import BaseModel

# Import the main RAG pipeline.
#
# answer_question() handles:
# - query rewriting
# - PDF retrieval
# - prompt building
# - Ollama generation
# - citation validation
from app.rag.pipeline import answer_question


# Create a FastAPI router.
#
# This router will contain the API endpoints
# related to the chatbot.
router = APIRouter()


class ChatMessage(BaseModel):
    """
```

```
    Represents one message in the conversation history.
    """

    # The role tells us who sent the message.
    #
    # Examples:
    # "user"
    # "assistant"
    role: str

    # The actual message content.
    content: str


class ChatRequest(BaseModel):
    """
    Represents the data that the frontend sends
    when asking YUMI a question.
    """

    # The user's current question.
    question: str

    # Previous conversation messages.
    #
    # Example:
    #
    # [
    #     {
    #         "role": "user",
    #         "content": "What is Sellio?"
    #     },
    #     {
    #         "role": "assistant",
    #         "content": "Sellio is..."
    #     }
    # ]
    #
    # If the frontend does not provide messages,
    # an empty list is used.
    messages: list[ChatMessage] = []


@router.post("/chat")
async def chat(request: ChatRequest):
    """
    Handle a chatbot request from the frontend.
    """

    # Send the user's question and conversation history
    # to the main RAG pipeline.
    #
    # request.question:
    #     The user's current question.
    #
    # request.messages:
    #     Previous conversation messages.
    #
    # m.model_dump():
    #     Converts each Pydantic ChatMessage object
    #     into a normal Python dictionary.
    return await answer_question(
        request.question,
        [
            m.model_dump()
            for m in request.messages
        ],
    )
```

backend/app/rag/pdf_extractor.py

```
# Import the dataclass decorator.
# @dataclass makes it easier to create classes that mainly store data.
from dataclasses import dataclass

# Import PyMuPDF.
# "fitz" is the Python module used to open and read PDF files.
import fitz


# This class represents the information extracted from ONE PDF page.
@dataclass
class PdfPage:
    # Stores the page number.
    # int means the value should be an integer, such as 1, 2, 3, etc.
    page: int

    # Stores the text extracted from this page.
    # str means the value is a string.
    text: str
```

```
# This class represents the COMPLETE result after processing a PDF.
@dataclass
class PdfResult:
    # Stores all the extracted text from the entire PDF
    # as one large string.
    text: str

    # Stores the total number of pages in the PDF.
    pages: int

    # Stores the extracted information for every page.
    # Each item in this list is a PdfPage object.
    page_texts: list[PdfPage]


# Function that extracts text from a PDF file.
#
# file_path: str
#   The path/location of the PDF file.
#
# -> PdfResult
#   Means this function returns a PdfResult object.
def extract_pdf_text(file_path: str) -> PdfResult:

    # Open the PDF file using PyMuPDF.
    #
    # fitz.open() loads the PDF so we can access
    # its pages and extract their text.
    document = fitz.open(file_path)

    # Create an empty list.
    #
    # We will store the extracted information
    # for every PDF page inside this list.
    pages = []

    # Loop through every page in the PDF.
    #
    # enumerate() gives us two things:
    #   index -> the number/index of the page, starting from 0
    #   page  -> the actual PDF page object
    for index, page in enumerate(document):

        # Extract text from the current PDF page.
        #
        # page.get_text("text")
        # tells PyMuPDF that we want the text content.
        #
        # "or ''" means:
        # If no text is returned, use an empty string instead.
        text = page.get_text("text") or ""

        # Create a PdfPage object containing:
        #
        # index + 1 -> the human-readable page number
        # text      -> the text extracted from that page
        #
        # Python starts counting from 0, so we add 1:
        # 0 -> Page 1
        # 1 -> Page 2
        # 2 -> Page 3
        #
        # append() adds the PdfPage object to our pages list.
        pages.append(PdfPage(index + 1, text))

    # Close the PDF after we have finished reading it.
    #
    # This releases the file/resource that was being used.
    document.close()

    # Combine the text from all PDF pages into one large string.
    #
    # p.text for p in pages
    # gets the text from each PdfPage object.
    #
    # "\n\n".join(...)
    # puts two newline characters between each page's text.
    #
    # Example:
    #
    # Page 1 text
    #
    # Page 2 text
    #
    # Page 3 text
    full_text = "\n\n".join(p.text for p in pages)

    # Return a PdfResult containing all the information
    # we collected from the PDF.
    return PdfResult(
        # The complete text from the entire PDF.
        text=full_text,
```

```
        # Count how many pages were extracted.
        pages=len(pages),

        # Store the list containing each page's information.
        page_texts=pages,
    )
```

backend/app/rag/chunker.py

```
# Define a function called chunk_text.
#
# text: str
#     The text that we want to split into smaller pieces.
#
# chunk_size: int = 1500
#     The maximum number of characters allowed in each chunk.
#     If the user does not provide a value, 1500 is used.
#
# overlap: int = 200
#     The number of characters that should overlap between
#     consecutive chunks.
#     If the user does not provide a value, 200 is used.
#
# -> list[str]
#     The function returns a list of strings.
#     Each string represents one text chunk.
def chunk_text(
    text: str,
    chunk_size: int = 1500,
    overlap: int = 200
) -> list[str]:

    # Check whether the text is empty or contains only whitespace.
    #
    # text.strip()
    # removes spaces, tabs, and newlines from the beginning
    # and end of the text.
    #
    # If nothing remains, the text is considered empty.
    #
    # Example:
    #     "   " -> ""
    #
    # If the text is empty, return an empty list.
    if not text.strip():
        return []

    # Make sure the overlap is smaller than the chunk size.
    #
    # For example:
    #     chunk_size = 1500
    #     overlap = 200
    #
    # This is valid because 200 < 1500.
    #
    # But:
    #     chunk_size = 1500
    #     overlap = 1500
    #
    # would be invalid because the chunks would not move forward.
    if overlap >= chunk_size:
        raise ValueError("overlap must be smaller than chunk_size")

    # Create an empty list where we will store
    # all of the generated text chunks.
    chunks = []

    # Start reading the text from character position 0.
    #
    # This represents the starting position of our first chunk.
    start = 0

    # Continue creating chunks as long as the starting
    # position is still inside the text.
    while start < len(text):

        # Calculate where the current chunk should end.
        #
        # start + chunk_size
        # gives us the normal ending position.
        #
        # min() makes sure we don't go beyond the end
        # of the text.
        #
        # Example:
        #     text length = 4000
        #     start = 0
        #     chunk_size = 1500
        #
        # end = 1500
        end = min(start + chunk_size, len(text))
```

```
        # Extract the text between start and end.
        #
        # .strip()
        # removes unnecessary whitespace from the beginning
        # and end of the chunk.
        chunk = text[start:end].strip()

        # Make sure the chunk is not empty.
        #
        # If the chunk contains actual text,
        # add it to the chunks list.
        if chunk:
            chunks.append(chunk)

        # Move the starting position forward.
        #
        # We don't move by the entire chunk_size because
        # we want some text from the previous chunk
        # to appear again in the next chunk.
        #
        # Example:
        #     chunk_size = 1500
        #     overlap = 200
        #
        # start moves by:
        #
        #     1500 - 200 = 1300
        #
        # So the next chunk starts 1300 characters later,
        # leaving 200 characters of overlap.
        start += chunk_size - overlap

    # Return the complete list of generated chunks.
    return chunks
```

backend/app/rag/embeddings.py

```
# Import lru_cache from Python's functools module.
#
# lru_cache allows us to save/cache the result of a function
# so that we don't have to run the same expensive operation
# repeatedly.
from functools import lru_cache


# Import SentenceTransformer.
#
# SentenceTransformer is used to convert text into
# numerical vectors called "embeddings".
#
# These embeddings allow us to compare the meaning
# of different pieces of text.
from sentence_transformers import SentenceTransformer


# Name of the AI embedding model that we want to use.
#
# all-MiniLM-L6-v2 is a relatively small and commonly used
# Sentence Transformer model for generating text embeddings.
#
# The model converts text into a numerical vector.
MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"


# Cache the model so it only needs to be loaded once.
#
# maxsize=1 means we keep a maximum of one result
# in the cache.
#
# Loading an AI model can be expensive and slow,
# so we don't want to load the model every time
# get_model() is called.
@lru_cache(maxsize=1)
def get_model():

    # Load the Sentence Transformer model.
    #
    # MODEL_NAME contains:
    # "sentence-transformers/all-MiniLM-L6-v2"
    #
    # The first time this function runs, the model is loaded.
    #
    # After that, because of @lru_cache, Python can reuse
    # the already-loaded model instead of loading it again.
    return SentenceTransformer(MODEL_NAME)


# Create an embedding for ONE piece of text.
#
# text: str
#     The text we want to convert into an embedding.
```

```
#
# -> list[float]
#     The function returns a list of floating-point numbers.
def create_embedding(text: str) -> list[float]:

    # Get the already-loaded Sentence Transformer model.
    #
    # Then use encode() to convert the text into
    # a numerical vector.
    vector = get_model().encode(

        # The actual text that we want to convert.
        text,

        # Normalize the resulting vector.
        #
        # Normalization makes the vector have a length of 1.
        #
        # This is useful when comparing embeddings using
        # similarity methods such as cosine similarity.
        normalize_embeddings=True,
    )

    # Convert the NumPy array returned by encode()
    # into a normal Python list.
    #
    # Example:
    #
    # NumPy array:
    # [0.12, -0.45, 0.78, ...]
    #
    # becomes:
    #
    # [0.12, -0.45, 0.78, ...]
    #
    # but as a normal Python list.
    return vector.tolist()


# Create embeddings for MULTIPLE pieces of text at once.
#
# texts: list[str]
#     A list containing multiple text strings.
#
# Example:
#
# texts = [
#     "What is blockchain?",
#     "What is PostgreSQL?",
#     "What is React?"
# ]
#
# The function generates one embedding for each text.
def create_embeddings(texts: list[str]):

    # Get the cached Sentence Transformer model
    # and encode all the texts at once.
    #
    # Encoding multiple texts together is generally
    # more efficient than calling create_embedding()
    # repeatedly for every piece of text.
    vectors = get_model().encode(

        # The list of text strings to convert into embeddings.
        texts,

        # Normalize every generated embedding.
        normalize_embeddings=True,
    )

    # Convert the returned NumPy array into a normal
    # Python list.
    #
    # Because we supplied multiple texts, this will be
    # a list containing multiple embedding vectors.
    return vectors.tolist()
```

backend/app/rag/ingestion.py

```
# Import Path from pathlib.
#
# Path makes it easier and safer to work with file paths
# across different operating systems.
from pathlib import Path


# Import create_engine and text from SQLAlchemy.
#
# create_engine() creates a connection to the database.
#
# text() allows us to write raw SQL queries that can
# be executed through SQLAlchemy.
```

```
from sqlalchemy import create_engine, text


# Import environ from os.
#
# environ allows us to access environment variables,
# such as DATABASE_URL.
from os import environ


# Import load_dotenv.
#
# This loads variables from a .env file into the
# application's environment.
from dotenv import load_dotenv


# Import our PDF extraction function.
#
# This function was created earlier and is responsible
# for extracting text from the PDF.
from app.rag.pdf_extractor import extract_pdf_text


# Import our text chunking function.
#
# This function takes large text and divides it
# into smaller chunks.
from app.rag.chunker import chunk_text


# Import our embedding function.
#
# This converts each text chunk into a numerical vector
# that can be used for semantic/vector searching.
from app.rag.embeddings import create_embedding


# Load the variables from the .env file.
#
# For example, your .env file may contain:
#
# DATABASE_URL=postgresql://user:password@host/database
#
# After load_dotenv(), Python can access DATABASE_URL
# through environ["DATABASE_URL"].
load_dotenv()


# Create a SQLAlchemy database engine.
#
# environ["DATABASE_URL"]
# gets the database connection URL from the environment.
#
# pool_pre_ping=True
# tells SQLAlchemy to check whether a database connection
# is still alive before using it.
#
# This helps prevent errors caused by stale database
# connections.
engine = create_engine(
    environ["DATABASE_URL"],
    pool_pre_ping=True,
)


# Define the function that handles the complete PDF ingestion process.
#
# file_path:
#     The location of the PDF file on the computer/server.
#
# file_name:
#     Optional name of the PDF.
#     None means the user does not have to provide it.
#
# user_id:
#     Optional ID of the user who uploaded the PDF.
#
# -> int
#     The function returns an integer representing
#     the total number of chunks inserted into the database.
def ingest_pdf(
    file_path: str,
    file_name: str | None = None,
    user_id: str | None = None,
) -> int:

    # Convert the file path string into a Path object.
    #
    # Path gives us useful methods such as:
    #     path.exists()
    #     path.name
```

```
#
# Example:
#
# "uploads/document.pdf"
#
# becomes a Path object representing that location.
path = Path(file_path)


# Check whether the PDF file actually exists.
#
# If the file does not exist, stop the function
# and raise an error.
if not path.exists():
    raise FileNotFoundError(path)


# If file_name was provided, use it.
#
# If file_name is None or empty, use the actual
# filename from the Path object.
#
# Example:
#
# file_path = "/uploads/my_document.pdf"
#
# path.name = "my_document.pdf"
file_name = file_name or path.name


# Extract all text from the PDF.
#
# extract_pdf_text() returns a PdfResult containing:
#
#     result.text
#         -> all PDF text
#
#     result.pages
#         -> number of pages
#
#     result.page_texts
#         -> individual page information
result = extract_pdf_text(str(path))


# Start a database transaction.
#
# engine.begin() automatically:
#
#     commits if everything succeeds
#
#     rolls back if an error occurs
#
# "connection" is the active database connection
# used for our SQL queries.
with engine.begin() as connection:


    # Delete an existing PDF history record
    # with the same filename.
    #
    # This prevents multiple old versions of the same
    # filename from remaining in pdf_history.
    #
    # :file_name is a parameter placeholder.
    # The actual value is safely provided separately
    # in the dictionary below.
    connection.execute(
        text("""
            DELETE FROM pdf_history
            WHERE file_name = :file_name
        """),
        {
            "file_name": file_name,
        },
    )


    # Insert a new record into the pdf_history table.
    #
    # pdf_history stores information about the uploaded
    # and extracted PDF.
    #
    # The RETURNING id part asks PostgreSQL to give us
    # the ID of the newly inserted record.
    history_id = connection.execute(
        text("""
            INSERT INTO pdf_history (
                user_id,
                file_name,
                extracted_text
            )
```

```
            VALUES (
                :user_id,
                :file_name,
                :extracted_text
            )
            RETURNING id
        """),
        {
            # The ID of the user who uploaded the PDF.
            "user_id": user_id,

            # The name of the PDF file.
            "file_name": file_name,

            # The complete extracted text from the PDF.
            "extracted_text": result.text,
        },
    ).scalar_one()


    # Keep track of how many chunks we successfully
    # insert into the database.
    #
    # It starts at zero.
    total = 0


    # Loop through every page extracted from the PDF.
    #
    # result.page_texts contains PdfPage objects.
    #
    # Each page contains:
    #
    #     page.page
    #     page.text
    for page in result.page_texts:


        # Split the current page's text into smaller chunks.
        #
        # chunk_text() returns something like:
        #
        # [
        #     "First 1500 characters...",
        #     "Next chunk...",
        #     "Next chunk..."
        # ]
        for index, chunk in enumerate(
            chunk_text(page.text)
        ):


            # Convert the current text chunk into
            # a numerical embedding vector.
            #
            # Example:
            #
            # "Blockchain is..."
            #
            # becomes something conceptually like:
            #
            # [0.12, -0.45, 0.78, ...]
            vector = create_embedding(chunk)


            # Insert the current chunk and its embedding
            # into the pdf_chunk table.
            #
            # CAST(:embedding AS vector)
            # tells PostgreSQL to treat the embedding
            # as a pgvector vector type.
            connection.execute(
                text("""
                    INSERT INTO pdf_chunk (
                        pdf_history_id,
                        text,
                        page,
                        chunk_index,
                        embedding
                    )
                    VALUES (
                        :history_id,
                        :text,
                        :page,
                        :chunk_index,
                        CAST(:embedding AS vector)
                    )
                """),
                {
                    # ID of the PDF history record we created
                    # earlier.
                    "history_id": history_id,
```

```
                        # The actual text of this chunk.
                        "text": chunk,

                        # The page number where this chunk came from.
                        "page": page.page,

                        # The chunk's position within that page.
                        #
                        # enumerate() starts at 0:
                        #
                        # 0 = first chunk
                        # 1 = second chunk
                        # 2 = third chunk
                        "chunk_index": index,

                        # Convert the Python embedding list into
                        # a string representation that PostgreSQL
                        # pgvector can cast into a vector.
                        "embedding": str(vector),
                    },
                )


                # Increase the total chunk counter by one
                # after successfully inserting the chunk.
                total += 1


    # Return the total number of chunks inserted.
    #
    # Example:
    #
    # If the PDF produced 25 chunks:
    #
    #     return 25
    return total
```

backend/app/rag/retrieval.py

```
# Import create_engine and text from SQLAlchemy.
#
# create_engine() creates the connection to PostgreSQL.
# text() allows us to execute SQL queries.
from sqlalchemy import create_engine, text


# Import environ so we can read environment variables,
# such as DATABASE_URL.
from os import environ


# Import load_dotenv so we can load variables
# from the .env file.
from dotenv import load_dotenv


# Import the function that converts text into
# an embedding vector.
#
# We previously created this function in embeddings.py.
from app.rag.embeddings import create_embedding


# Load the variables stored inside the .env file.
#
# For example:
#
# DATABASE_URL=postgresql://...
#
# After this runs, Python can access DATABASE_URL
# using environ["DATABASE_URL"].
load_dotenv()


# Create a SQLAlchemy database engine.
#
# DATABASE_URL contains the connection information
# for your PostgreSQL database.
#
# pool_pre_ping=True checks whether a database connection
# is still alive before SQLAlchemy uses it.
engine = create_engine(
    environ["DATABASE_URL"],
    pool_pre_ping=True,
)


# Define a function that searches for PDF chunks
# relevant to a user's question.
#
# question:
```

```
#     The question asked by the user.
#
# limit:
#     Maximum number of relevant chunks that we want
#     to return.
#
#     Default = 5
#
# max_distance:
#     Maximum allowed vector distance.
#
#     Smaller distance generally means the vectors
#     are more similar.
#
#     Default = 0.65
def retrieve_relevant_chunks(
    question: str,
    limit: int = 5,
    max_distance: float = 0.65,
):

    # Convert the user's question into an embedding.
    #
    # Example:
    #
    # "What is blockchain?"
    #
    # becomes a numerical vector such as:
    #
    # [0.12, -0.34, 0.56, ...]
    #
    # str() converts the vector into a string representation
    # that can be passed to PostgreSQL.
    vector = str(create_embedding(question))


    # Start a database transaction/connection.
    #
    # The connection will automatically be handled
    # by the context manager.
    with engine.begin() as connection:


        # Execute a PostgreSQL query to find PDF chunks
        # that are semantically similar to the question.
        rows = connection.execute(
            text("""
                SELECT
                    c.id,
                    c.pdf_history_id,
                    c.text,
                    c.page,
                    c.chunk_index,

                    -- Calculate the vector distance between
                    -- the stored PDF chunk embedding and
                    -- the question embedding.
                    -- <=> is the pgvector distance operator.
                    c.embedding <=> CAST(:vector AS vector)
                        AS distance,

                    -- Get the original PDF filename.
                    h.file_name

                FROM pdf_chunk c

                -- Connect each chunk to its PDF history record.
                -- This allows us to know which PDF the chunk came from.
                JOIN pdf_history h
                    ON c.pdf_history_id = h.id

                -- Ignore chunks that don't have an embedding.
                WHERE c.embedding IS NOT NULL

                -- Only keep chunks whose vector distance is within the maximum.
                AND c.embedding <=> CAST(:vector AS vector)
                    <= :max_distance

                -- Sort from smallest distance to largest; smaller means more similar.
                ORDER BY c.embedding <=> CAST(:vector AS vector)

                -- Retrieve extra candidates because duplicates are removed later.
                LIMIT 10
            """),
            {
                # Send the question's embedding to PostgreSQL.
                "vector": vector,

                # Send the maximum allowed distance.
                "max_distance": max_distance,
            },
        ).mappings().all()
```

```
# Create an empty list.
#
# This will contain the final selected chunks
# after duplicate chunks have been removed.
selected = []


# Go through each candidate returned by PostgreSQL.
for row in rows:

    # Convert the current chunk's text into a set
    # of lowercase words.
    #
    # lower()
    # makes the comparison case-insensitive.
    #
    # split()
    # separates the text into individual words.
    #
    # set()
    # removes duplicate words.
    words = set(
        row["text"].lower().split()
    )


    # Assume this chunk is not a duplicate.
    duplicate = False


    # Compare the current chunk with every chunk
    # that we have already selected.
    for existing in selected:

        # Convert the already-selected chunk into
        # a set of lowercase words.
        other = set(
            existing["text"].lower().split()
        )


        # Calculate how similar the two chunks are
        # based on their shared words.
        #
        # words & other
        # gives the words that appear in BOTH chunks.
        #
        # words | other
        # gives all unique words appearing in either chunk.
        #
        # This formula is called Jaccard similarity:
        #
        #     intersection / union
        #
        # If the result is >= 0.80,
        # the chunks are considered duplicates.
        if (
            words
            and other
            and len(words & other) / len(words | other) >= 0.80
        ):

            # Mark this chunk as a duplicate.
            duplicate = True

            # No need to compare it with the remaining
            # selected chunks.
            break


    # Only add the chunk if it wasn't considered
    # a duplicate.
    if not duplicate:

        # Convert the SQLAlchemy row into a normal
        # Python dictionary and add it to selected.
        selected.append(dict(row))


    # Stop once we have collected the requested
    # number of chunks.
    #
    # For example:
    #
    # limit = 5
    #
    # Once selected contains 5 chunks,
    # we stop searching.
    if len(selected) >= limit:
        break
```

```
    # Return the final list of relevant PDF chunks.
    return selected
```

backend/app/rag/query_rewriter.py

```
# Import Python's regular expression module.
#
# The re module allows us to search for patterns
# inside text.
import re


# Create a list of regular expression patterns.
#
# These patterns are used to detect whether a user's
# question looks like a follow-up question.
#
# For example:
#
#     "Why is it important?"
#     "How does this work?"
#     "What about that?"
#
# These questions can be difficult to understand by
# themselves because words like "it", "this", or "that"
# refer to something mentioned previously.
PATTERNS = [

    # Detect questions that start with:
    #
    # why, how, when, or where
    #
    # and later contain a reference word such as:
    #
    # it, this, that, they, these, those
    #
    # Example:
    #
    # "Why is it important?"
    # "How does this work?"
    r"^(why|how|when|where) .*\b(it|this|that|they|these|those)\b",


    # Detect questions beginning with:
    #
    # what, which, or who
    #
    # followed somewhere by a reference word.
    #
    # Example:
    #
    # "What does this mean?"
    # "Which one is it?"
    r"^(what|which|who) .*\b(it|this|that|they|these|those)\b",


    # Detect questions such as:
    #
    # "What about this?"
    # "How about that?"
    # "Why about..."
    #
    # The pattern specifically looks for:
    #
    # what/how/why + about
    r"^(what|how|why) about\b",


    # Detect short follow-up questions beginning with:
    #
    # and what...
    # but how...
    # so why...
    #
    # Example:
    #
    # "And what about this?"
    # "But how does it work?"
    r"^(and|but|so) (what|how|why)\b",


    # Detect questions such as:
    #
    # "Explain it."
    # "Elaborate on this."
    # "Clarify that."
    # "Expand on it."
    #
    # These clearly refer to something from
    # the previous conversation.
```

```
    r"^(explain|elaborate|clarify|expand on) (it|this|that)\b",
]


# Define a function that determines whether a question
# is probably a follow-up question.
#
# question: str
#     The user's current question.
#
# -> bool
#     Returns either True or False.
#
# True  = looks like a follow-up question
# False = looks like a standalone question
def is_follow_up_question(question: str) -> bool:

    # Convert the question to lowercase and remove
    # unnecessary whitespace.
    #
    # Example:
    #
    # "  WHY IS IT IMPORTANT?  "
    #
    # becomes:
    #
    # "why is it important?"
    #
    # lower()
    # converts uppercase letters to lowercase.
    #
    # strip()
    # removes spaces from the beginning and end.
    q = question.lower().strip()


    # Remove punctuation marks from the END of the question.
    #
    # The regular expression:
    #
    # [.!?]+$
    #
    # means:
    #
    # . ! ?
    # one or more times
    # at the end of the string
    #
    # Example:
    #
    # "Why is it important???"
    #
    # becomes:
    #
    # "why is it important"
    q = re.sub(
        r"[.!?]+$",
        "",
        q,
    )


    # Check every pattern in PATTERNS.
    #
    # re.search() looks for the pattern inside q.
    #
    # any() returns True as soon as at least one
    # pattern matches.
    #
    # Therefore:
    #
    # True  -> the question looks like a follow-up
    # False -> none of the patterns matched
    return any(
        re.search(pattern, q)
        for pattern in PATTERNS
    )


# Define a function that rewrites a follow-up question
# using the previous conversation.
#
# question:
#     The user's current question.
#
# messages:
#     A list of previous conversation messages.
#
# -> str
#     Returns a question that includes the previous
#     user question when necessary.
def rewrite_query(
```

```
    question: str,
    messages: list[dict],
) -> str:

    # If there are no previous messages,
    # there is nothing to use as context.
    #
    # OR:
    #
    # If the current question is NOT a follow-up,
    # there is no need to rewrite it.
    #
    # In either situation, return the original question.
    if not messages or not is_follow_up_question(question):
        return question


    # Get the content of previous messages where
    # the role is "user".
    #
    # This filters out assistant messages.
    #
    # Example messages:
    #
    # [
    #     {"role": "user", "content": "What is blockchain?"},
    #     {"role": "assistant", "content": "Blockchain is..."},
    #     {"role": "user", "content": "Why is it secure?"}
    # ]
    #
    # The result becomes:
    #
    # [
    #     "What is blockchain?",
    #     "Why is it secure?"
    # ]
    previous = [
        m["content"]
        for m in messages
        if m.get("role") == "user"
    ]


    # If there are no previous user messages,
    # return the original question.
    if not previous:
        return question


    # Combine the most recent previous user question
    # with the current follow-up question.
    #
    # previous[-1]
    # means the LAST item in the previous list.
    #
    # Example:
    #
    # Previous question:
    # "What is blockchain?"
    #
    # Current question:
    # "Why is it secure?"
    #
    # The result becomes:
    #
    # "What is blockchain?
    #  Follow-up question: Why is it secure?"
    #
    # This gives the retrieval system enough context
    # to understand what "it" refers to.
    return (
        f"{previous[-1]}\n"
        f"Follow-up question: {question}"
    )
```

backend/app/rag/prompt.py

```
def build_rag_prompt(
    question: str,
    chunks: list[dict],
    history: list[dict] | None = None,
) -> str:
    # If no relevant PDF chunks were found,
    # immediately tell the chatbot to return this exact answer.
    if not chunks:
        return "Answer exactly: Information not found."

    # This list will contain the formatted information
    # retrieved from the PDF.
    context = []

    # Loop through every retrieved chunk.
```

```
    #
    # enumerate(..., 1) means the numbering starts at 1
    # instead of 0. This gives us:
    # Source 1, Source 2, Source 3, etc.
    for index, chunk in enumerate(chunks, 1):

        # Format each retrieved chunk so the AI knows:
        # - which source it came from
        # - which PDF file it came from
        # - which page it came from
        # - the actual text of the chunk
        context.append(
            f"[Source {index}] "
            f"File: {chunk['file_name']}; "
            f"Page: {chunk['page']}\n"
            f"{chunk['text']}"
        )

    # Start with an empty conversation history.
    history_text = ""

    # Only process the history if conversation messages exist.
    if history:

        # Take only the last 6 messages.
        #
        # history[-6:] means:
        # "Give me the most recent 6 messages."
        #
        # This prevents the prompt from becoming unnecessarily
        # large when the conversation gets longer.
        history_text = "\n".join(
            f"{m['role']}: {m['content']}"
            for m in history[-6:]
        )

    # Build the final prompt that will be sent to the AI model.
    return f"""
You are YUMI, a document question-answering assistant.

# Instructions

- Use only the supplied context.
- Do not invent facts.
- Be polite, warm, and direct. Answer the question first without a long preamble.
- Organize the reply for easy reading: use short paragraphs, and leave a blank
  line between separate ideas.
- For multiple projects, skills, or technologies, use a clear bullet list with
  one item per line. Keep each project and its details together.
- Avoid repeating the question, adding unsupported conclusions, or combining
  unrelated facts into one paragraph.
- Treat short fragments as direct questions. For example, "Tech stack" means
  "What technologies are used?"
- If the context contains the answer, answer directly and cite it at the end
  using the exact label shown in the context, such as [Source 1].
- Keep the reply concise. Do not start with phrases like "For the question" or
  "The explicitly named items are".
- For a question about technologies or a tech stack, list the technologies
  explicitly named in the context.
- For a broad portfolio tech-stack question, prefer the portfolio-wide
  "TECH STACKS" overview over stacks listed under individual projects.
- Group technologies by category, remove duplicates, and do not list connector
  words such as "and" as technologies.
- When asked to list projects, use the "PROJECTS" section and include every
  project name stated there. Do not refuse if the retrieved context lists them.
- For a list, use one bullet per distinct item, omit duplicates, and keep the
  wording parallel across items.
- If the context does not support the answer, return exactly:
  Information not found.

# Conversation

{history_text}

# Retrieved Context

{"\n\n".join(context)}

# Question

{question}

# Answer

""".strip()
```

backend/app/rag/llm.py

```
import os
import httpx
from dotenv import load_dotenv
```

```
# Load variables from the .env file.
#
# This allows us to store settings such as:
# OLLAMA_HOST=http://localhost:11434
# OLLAMA_MODEL=llama3.2:1b
load_dotenv()


# Get the Ollama server address from the .env file.
#
# If OLLAMA_HOST is not found,
# use http://localhost:11434 as the default.
OLLAMA_HOST = os.getenv(
    "OLLAMA_HOST",
    "http://localhost:11434",
)


# Get the AI model name from the .env file.
#
# If OLLAMA_MODEL is not found,
# use llama3.2:1b as the default model.
MODEL = os.getenv(
    "OLLAMA_MODEL",
    "llama3.2:1b",
)


async def generate_text(prompt: str) -> str:
    """
    Send a prompt to the Ollama AI model
    and return the generated answer.
    """

    # Create an asynchronous HTTP client.
    #
    # AsyncClient allows our application to communicate
    # with Ollama without blocking the entire application
    # while waiting for the AI response.
    #
    # Leave room for FastAPI to return an error before the Next.js proxy
    # times out. Model loading is warmed separately during setup/startup.
    timeout = httpx.Timeout(210.0, connect=10.0)
    async with httpx.AsyncClient(timeout=timeout) as client:

        # Send a POST request to Ollama's chat API.
        #
        # /api/chat is Ollama's endpoint for chat-based
        # text generation.
        response = await client.post(
            f"{OLLAMA_HOST}/api/chat",

            # Send the model and conversation to Ollama.
            json={
                # Tell Ollama which model should generate
                # the response.
                "model": MODEL,

                # Provide the prompt as a user message.
                #
                # The prompt here will normally be the
                # RAG prompt created by build_rag_prompt().
                "messages": [
                    {
                        "role": "user",
                        "content": prompt,
                    }
                ],

                # We want one complete response instead
                # of receiving the answer piece-by-piece.
                "stream": False,

                # Additional settings for the model.
                "options": {

                    # A smaller context is faster on CPU and sufficient
                    # for the question plus the retrieved PDF chunks.
                    "num_ctx": 4096,

                    # Keep answers concise so CPU generation completes sooner.
                    "num_predict": 128,
                },
                # Keep the small local model loaded until Ollama restarts,
                # avoiding multi-minute cold starts after idle periods.
                "keep_alive": -1,
            },
        )

        # Raise an exception if Ollama returned an HTTP error.
        #
```

```
        # For example:
        # 404 -> endpoint/model problem
        # 500 -> server-side problem
        response.raise_for_status()

        # Convert Ollama's JSON response into a Python dictionary.
        #
        # Then access:
        # response -> message -> content
        #
        # .strip() removes unnecessary whitespace
        # from the beginning and end of the answer.
        return response.json()["message"]["content"].strip()
```

backend/app/rag/pipeline.py

```
import re

# Rewrite follow-up questions so they contain enough context
# for the vector search.
from app.rag.query_rewriter import rewrite_query

# Search PostgreSQL/pgvector for relevant PDF chunks.
from app.rag.retrieval import retrieve_relevant_chunks

# Build the final prompt containing the instructions,
# conversation history, retrieved PDF context, and question.
from app.rag.prompt import build_rag_prompt

# Send the prompt to Ollama and generate the AI answer.
from app.rag.llm import generate_text


# This is the answer returned when the system cannot
# find enough information to answer the question.
FALLBACK = "Information not found."


def valid_citation(answer: str, count: int) -> bool:
    """
    Check whether the AI answer contains at least one
    valid [Source X] citation.
    """

    # Search the answer for citations such as:
    #
    # [Source 1]
    # [Source 2]
    # [Source 3]
    #
    # re.findall() returns only the number inside
    # the citation.
    #
    # Example:
    # "[Source 1] and [Source 3]"
    #
    # becomes:
    # ["1", "3"]
    numbers = [
        int(x)
        for x in re.findall(
            r"\[Source\s+(\d+)\]",
            answer,
            flags=re.I,
        )
    ]

    # Check whether at least one citation number
    # is within the valid source range.
    #
    # If there are 5 retrieved chunks, valid citations
    # are:
    #
    # [Source 1]
    # [Source 2]
    # [Source 3]
    # [Source 4]
    # [Source 5]
    return any(1 <= n <= count for n in numbers)


def format_project_stacks(chunks: list[dict]) -> str | None:
    """Extract each project's description and adjacent TECHSTACK USED list."""
    project_lines: list[str] = []

    for source_index, chunk in enumerate(chunks, 1):
        blocks = re.split(r"(?m)^\s*\d+\.\s+", chunk["text"])

        for block in blocks[1:]:
            first_line = block.strip().splitlines()[0].strip()
            # The PDF uses an en dash between each project name and its
            # description. Unicode escapes make this robust to file encodings.
```

```
            title_parts = re.split(r"\s+[\u2013\u2014]\s+|\s+-\s+", first_line, maxsplit=1)
            project_name = title_parts[0].strip(" -*")

            stack_match = re.search(
                r"TECH\s*STACK\s*USED\s*:\s*(.*)",
                block,
                flags=re.I | re.S,
            )
            if not stack_match:
                continue

            # Keep the descriptor after the dash and any following description
            # text that appears before the project's TECHSTACK USED heading.
            description_parts = []
            if len(title_parts) > 1 and title_parts[1].strip():
                description_parts.append(title_parts[1].strip(" -*:."))
            description_lines = block[: stack_match.start()].splitlines()[1:]
            description_parts.extend(
                line.strip(" -*\t")
                for line in description_lines
                if line.strip(" -*\t")
            )
            description = re.sub(r"\s+", " ", " ".join(description_parts)).strip()

            stack_text = stack_match.group(1)
            technologies = [
                item.strip(" .;\n\r\t")
                for item in re.split(r",|\band\b", stack_text, flags=re.I)
                if item.strip(" .;\n\r\t")
            ]

            if technologies:
                project_line = f"- {project_name}"
                if description:
                    project_line += f"\n  About: {description}"
                project_line += (
                    f"\n  Tech stack: {', '.join(technologies)}"
                    f" [Source {source_index}]"
                )
                project_lines.append(project_line)

    if not project_lines:
        return None

    return "Projects, descriptions, and tech stacks:\n\n" + "\n\n".join(project_lines)


async def answer_question(
    question: str,
    messages: list[dict] | None = None,
):
    """
    Process a user's question through the complete
    RAG pipeline and return the final answer.
    """

    # If no conversation history was provided,
    # use an empty list instead of None.
    #
    # This makes it easier to work with messages later.
    messages = messages or []

    # Greetings are conversational and do not need a PDF search or an LLM call.
    # Use a full match so a real question such as "Hi, what projects..." still
    # continues through the normal RAG pipeline.
    normalized_question = re.sub(r"[!?.\s]+$", "", question.strip().casefold())
    if re.fullmatch(
        r"(?:hi|hello|hey)(?:\s+yumi)?|good\s+(?:morning|afternoon|evening)",
        normalized_question,
    ):
        return {
            "answer": "Hi! I’m YUMI. Ask me about Kemuel’s projects, skills, or experience.",
            "sources": [],
        }
    if re.fullmatch(
        r"(?:thank\s+you|thanks|thx)(?:\s+(?:yumi|so\s+much|a\s+lot))?",
        normalized_question,
    ):
        return {
            "answer": "You’re welcome! Feel free to ask if you’d like to know more.",
            "sources": [],
        }

    # ----------------------------------------------------------
    # STEP 1: Rewrite the user's question
    # ----------------------------------------------------------
    #
    # This is especially useful for follow-up questions.
    #
    # Example:
    #
    # Previous:
```

```
# "What is Sellio?"
#
# User:
# "What are its features?"
#
# rewrite_query() can add the previous question
# so retrieval has more context.
search_query = rewrite_query(
    question,
    messages,
)

asks_for_project_list = bool(
    re.search(
        r"\b(?:list|name|which|what|all)\b.*\bprojects?\b|"
        r"\bprojects?\b.*\b(?:all|list|names?)\b",
        question,
        re.I,
    )
)
asks_for_project_stacks = asks_for_project_list and bool(
    re.search(
        r"\b(?:tech(?:nology)?\s*stack|stacks?|technologies)\b",
        question,
        re.I,
    )
)
if asks_for_project_list:
    # Match the document's explicit project heading and its project names.
    # This helps retrieve the projects page instead of the general overview.
    search_query = (
        "PROJECTS list of all project names and descriptions: "
        "EventSphere, Social Welfare System, ECA-IS"
    )

# A broad tech-stack question should find the portfolio overview section,
# rather than one project's individual stack. Project-specific questions
# keep their original query so retrieval can focus on that project.
asks_for_general_stack = bool(
    re.search(r"\b(?:tech(?:nology)?\s*stack|technologies)\b", question, re.I)
)
mentions_a_project = bool(
    re.search(
        r"\b(?:project|eventsphere|social welfare system|eca\s*[-\u2013\u2014]?\s*is|internship)\b",
        question,
        re.I,
    )
)
if asks_for_general_stack and not mentions_a_project and not asks_for_project_list:
    search_query = (
        "portfolio-wide technology stack overview: Frontend Development, "
        "Backend Development, Databases, Cloud and Hosting, DevOps and "
        "Deployment, and UI/UX Design"
    )


# ------------------------------------------------------------
# STEP 2: Search for relevant PDF chunks
# ------------------------------------------------------------
#
# The rewritten question is converted into an embedding
# inside retrieve_relevant_chunks().
#
# PostgreSQL + pgvector then searches for similar
# chunks from the uploaded PDF.
#
# limit=5:
# Return at most 5 relevant chunks.
#
# max_distance=0.65:
# Only accept chunks whose vector distance is
# 0.65 or lower.
chunks = retrieve_relevant_chunks(
    search_query,
    limit=5,
    max_distance=0.65,
)

# When the matching section is present, pass only that section to the
# model. Similar pages can otherwise make it mix the overview with
# project-specific stacks, or answer a project-list question from page 1.
if asks_for_project_list:
    project_section = [
        chunk
        for chunk in chunks
        if re.search(r"\bPROJECTS\s*:", chunk["text"], re.I)
    ]
    if project_section:
        chunks = project_section
elif asks_for_general_stack and not mentions_a_project:
    stack_overview = [
        chunk
```

```
            for chunk in chunks
            if re.search(r"\bTECH\s*STACKS\b", chunk["text"], re.I)
        ]
        if stack_overview:
            chunks = stack_overview


    # ---------------------------------------------------------
    # STEP 3: Check whether anything relevant was found
    # ---------------------------------------------------------
    #
    # If PostgreSQL did not find any relevant chunks,
    # there is no reliable PDF information to give
    # to the AI.
    if not chunks:
        return {
            "answer": FALLBACK,
            "sources": [],
        }

    if asks_for_project_stacks:
        structured_answer = format_project_stacks(chunks)
        if structured_answer:
            return {
                "answer": structured_answer,
                "sources": chunks,
            }


    # ---------------------------------------------------------
    # STEP 4: Build the RAG prompt
    # ---------------------------------------------------------
    #
    # This combines:
    #
    # - the user's original question
    # - retrieved PDF chunks
    # - conversation history
    # - instructions for YUMI
    # - source citation requirements
    #
    # Notice that we use the ORIGINAL question here,
    # not search_query.
    #
    # search_query is mainly for better retrieval.
    # The original question is what we want YUMI
    # to actually answer.
    prompt = build_rag_prompt(
        question,
        chunks,
        messages,
    )


    # ---------------------------------------------------------
    # STEP 5: Send the prompt to Ollama
    # ---------------------------------------------------------
    #
    # generate_text() sends the prompt to the local
    # Ollama model and returns the generated answer.
    answer = await generate_text(prompt)


    # ---------------------------------------------------------
    # STEP 6: Validate the AI response
    # ---------------------------------------------------------
    #
    # Keep the fallback only when the model returned no answer or explicitly
    # said the retrieved context does not contain the answer. If it answers
    # without a valid citation, add the top-ranked source below.
    answer = answer.strip()
    if (
        not answer
        or answer.rstrip(".! ").casefold()
        == FALLBACK.rstrip(".! ").casefold()
    ):
        return {
            "answer": FALLBACK,
            "sources": chunks,
        }

    # Source 1 is the closest matching chunk in the context. Small local
    # models sometimes answer from the supplied context but omit the citation;
    # add the top-ranked source instead of discarding an otherwise useful answer.
    if not valid_citation(answer, len(chunks)):
        answer = re.sub(
            r"\[Source\s+\d+\]",
            "",
            answer,
            flags=re.I,
        ).strip()
        answer = f"{answer} [Source 1]"
```

```
    # ---------------------------------------------------------
    # STEP 7: Return the final successful response
    # ---------------------------------------------------------
    #
    # The answer is the text generated by Ollama.
    #
    # sources contains the retrieved PDF chunks so
    # the frontend can display source information.
    return {
        "answer": answer,
        "sources": chunks,
    }
```

src/app/api/chat/route.ts

```
import { NextResponse } from "next/server"

export async function POST(request: Request) {
  const backendUrl = process.env.PYTHON_API_URL

  if (!backendUrl) {
    return NextResponse.json(
      { error: "PYTHON_API_URL is not configured." },
      { status: 500 },
    )
  }

  try {
    const body = await request.json()
    const response = await fetch(`${backendUrl}/api/chat`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(body),
    })

    const responseText = await response.text()

    if (!response.ok) {
      // Keep the backend's response in the Next.js terminal for debugging.
      console.error(
        `Python chat backend returned ${response.status}: ${responseText}`,
      )
      return NextResponse.json(
        { error: "The chat backend failed. Check its Docker logs for details." },
        { status: response.status },
      )
    }

    try {
      return NextResponse.json(JSON.parse(responseText), {
        status: response.status,
      })
    } catch {
      console.error("Python chat backend returned invalid JSON:", responseText)
      return NextResponse.json(
        { error: "The chat backend returned an invalid response." },
        { status: 502 },
      )
    }
  } catch (error) {
    console.error("Could not reach the Python chat backend:", error)
    return NextResponse.json(
      { error: "Could not reach the chat backend. Check that it is running." },
      { status: 502 },
    )
  }
}
```

src/components/FloatingChat.tsx

```
"use client"

import { type FormEvent, useEffect, useState } from "react"
import { X } from "lucide-react"
import { AnimatePresence, motion } from "framer-motion"

type ChatMessage = {
  role: "user" | "assistant"
  content: string
}

type ChatSource = {
  file_name: string
  page: number
}


export function FloatingChat() {
  const [isOpen, setIsOpen] = useState(false)
```

```
  const [bubbleMessage, setBubbleMessage] = useState("Hi! I'm YUMI")

  const [messages, setMessages] = useState<ChatMessage[]>([])
  const [input, setInput] = useState("")
  const [sources, setSources] = useState<ChatSource[]>([])
  const [isLoading, setIsLoading] = useState(false)
  const [error, setError] = useState("")


  useEffect(() => {
    const timer = window.setTimeout(() => setBubbleMessage("Ask me!"), 3000)
    return () => window.clearTimeout(timer)
  }, [])

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault()

    const question = input.trim()
    if (!question || isLoading) return

    setInput("")
    setError("")
    setSources([])
    setIsLoading(true)

    try {
      const response = await fetch("/api/chat", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ question, messages }),
      })

      const data: {
        answer?: string
        sources?: ChatSource[]
        error?: string
      } = await response.json()

      if (!response.ok) {
        throw new Error(data.error ?? "The chat request failed.")
      }

      const answer = data.answer
      if (typeof answer !== "string" || !answer.trim()) {
        throw new Error("The chat backend returned no answer.")
      }

      setMessages((current) => [
        ...current,
        { role: "user", content: question },
        { role: "assistant", content: answer },
      ])
      setSources(data.sources ?? [])
    } catch (caughtError) {
      setError(
        caughtError instanceof Error
          ? caughtError.message
          : "Sorry, I couldn't get an answer. Please try again.",
      )
    } finally {
      setIsLoading(false)
    }
  }

  return (
    <motion.div
      initial={{ opacity: 0, y: 10 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.45, delay: 0.2, ease: "easeOut" }}
      className="fixed bottom-6 right-6 z-[9999] flex flex-col items-end gap-3"
    >
      <AnimatePresence>
        {isOpen && (
        <motion.section
          key="yumi-chat-panel"
          aria-label="YUMI AI chat"
          initial={{ opacity: 0, y: 12, scale: 0.97 }}
          animate={{ opacity: 1, y: 0, scale: 1 }}
          exit={{ opacity: 0, y: 8, scale: 0.97 }}
          transition={{ duration: 0.2, ease: "easeOut" }}
          className="mb-3 w-[min(20rem,calc(100vw-3rem))] rounded-2xl border border-card-border bg-background
    p-5 text-foreground shadow-xl"
        >
          <div className="mb-4 flex items-start justify-between">
            <div>
              <h2 className="font-semibold">Hi, I’m Yumi</h2>
              <p className="mt-1 text-sm text-secondary">
                What would you like to explore?
              </p>
            </div>

            <button
```

```
            type="button"
            onClick={() => setIsOpen(false)}
            aria-label="Close chat"
            className="rounded-full p-2 transition hover:bg-accent"
          >
            <X size={18} />
          </button>
        </div>

        <div
          aria-live="polite"
          className="max-h-64 space-y-3 overflow-y-auto"
        >
          {messages.length === 0 && (
            <p className="text-sm text-secondary">
              Ask me about my projects, skills, or experience.
            </p>
          )}

          {messages.map((message, index) => (
            <p
              key={`${message.role}-${index}`}
              className={
                message.role === "user"
                  ? "ml-6 rounded-2xl bg-accent p-3 text-sm leading-6"
                  : "mr-6 whitespace-pre-line rounded-2xl border border-card-border p-3 text-sm leading-6"
              }
            >
              {message.role === "assistant" && message.content.includes("\n- ")
                ? message.content.split("\n").map((line, lineIndex) => {
                    const isListItem = line.startsWith("- ")
                    return (
                      <span
                        key={lineIndex}
                        className={
                          isListItem
                            ? "block border-t border-card-border py-2 first:border-0"
                            : "block"
                        }
                      >
                        {line}
                      </span>
                    )
                  })
                : message.content}
            </p>
          ))}

          {isLoading && (
            <p className="text-sm text-secondary">YUMI is thinking…</p>
          )}

          {error && (
            <p role="alert" className="text-sm text-red-600">
              {error}
            </p>
          )}

          {sources.length > 0 && (
            <p className="text-xs text-secondary">
              Sources: {sources
                .map((source) => `${source.file_name}, page ${source.page}`)
                .join("; ")}
            </p>
          )}
        </div>

        <form onSubmit={handleSubmit} className="mt-4 flex gap-2">
          <input
            value={input}
            onChange={(event) => setInput(event.target.value)}
            placeholder="Ask YUMI a question..."
            aria-label="Your message"
            className="min-w-0 flex-1 rounded-xl border border-card-border bg-background px-3 py-2 text-sm
text-foreground"
          />
          <button
            type="submit"
            disabled={isLoading || !input.trim()}
            className="rounded-xl bg-primary px-4 py-2 text-sm text-background disabled:opacity-50"
          >
            Send
          </button>
        </form>
      </motion.section>
      )}
    </AnimatePresence>
    <div className="flex items-center gap-3">
      <AnimatePresence>
        {!isOpen && (
        <motion.span
          key="ask-bubble"
```

```
            initial={{ opacity: 0, x: 8, scale: 0.96 }}
            animate={{ opacity: 1, x: 0, scale: 1 }}
            exit={{ opacity: 0, x: 8, scale: 0.96 }}
            transition={{ duration: 0.18, ease: "easeOut" }}
            className="relative z-10 rounded-2xl bg-background px-4 py-2 text-sm font-medium text-foreground
    shadow-lg"
          >
            <AnimatePresence mode="wait" initial={false}>
              <motion.span
                key={bubbleMessage}
                initial={{ opacity: 0, y: 5 }}
                animate={{ opacity: 1, y: 0 }}
                exit={{ opacity: 0, y: -5 }}
                transition={{ duration: 0.25, ease: "easeOut" }}
                className="inline-block"
              >
                {bubbleMessage}
              </motion.span>
            </AnimatePresence>
            <span
              aria-hidden="true"
              className="absolute -right-1.5 top-1/2 h-3 w-3 -translate-y-1/2 rotate-45 bg-background"
            />
          </motion.span>
          )}
        </AnimatePresence>

        <div className="relative">
          {!isOpen && (
            <span
              aria-hidden="true"
              className="absolute inset-0 rounded-full border border-zinc-500/60 bg-zinc-400/20
    motion-safe:animate-ping"
            />
          )}
          <motion.button
            type="button"
            onClick={() => setIsOpen((open) => !open)}
            aria-label={isOpen ? "Close chat" : "Open chat"}
            aria-expanded={isOpen}
            whileHover={{ scale: 1.06 }}
            whileTap={{ scale: 0.92 }}
            transition={{ type: "spring", stiffness: 400, damping: 18 }}
            className="relative flex h-14 w-14 items-center justify-center rounded-full border border-zinc-200
    bg-white text-zinc-900 shadow-lg shadow-black/15 transition hover:bg-zinc-50 focus-visible:outline-none
    focus-visible:ring-4 focus-visible:ring-zinc-300"
          >
            <AnimatePresence mode="wait" initial={false}>
              {isOpen ? (
                <motion.span
                  key="close-icon"
                  initial={{ opacity: 0, rotate: -45, scale: 0.7 }}
                  animate={{ opacity: 1, rotate: 0, scale: 1 }}
                  exit={{ opacity: 0, rotate: 45, scale: 0.7 }}
                  transition={{ duration: 0.16 }}
                  className="flex"
                >
                  <X size={24} />
                </motion.span>
              ) : (
                <motion.span
                  key="ai-badge"
                  initial={{ opacity: 0, rotate: 20, scale: 0.7 }}
                  animate={{ opacity: 1, rotate: 0, scale: 1 }}
                  exit={{ opacity: 0, rotate: -20, scale: 0.7 }}
                  transition={{ duration: 0.16 }}
                  className="relative flex items-center justify-center font-black leading-none
    tracking-[-0.08em]"
                >
                  <span className="text-[17px] text-zinc-900">A</span>
                  <span className="text-[17px] text-zinc-500">I</span>
                  <span
                    aria-hidden="true"
                    className="absolute -right-1 -top-1 h-2 w-2 rounded-full border border-white bg-zinc-400
    shadow-sm"
                  />
                </motion.span>
              )}
            </AnimatePresence>
          </motion.button>
        </div>
      </div>
    </motion.div>
  )
}
```

scripts/Test-YumiRag.ps1

```
param(
    [string]$BackendUrl = "http://localhost:8000",
    [string]$NextUrl = "http://localhost:3000",
    [string]$OllamaUrl = "http://localhost:11434",
```

```
    [string]$ModelName = "llama3.2:1b"
)

$ErrorActionPreference = "Stop"
$script:passed = 0

function Write-Pass([string]$Message) {
    $script:passed++
    Write-Host "PASS: $Message" -ForegroundColor Green
}

function Invoke-YumiChat([string]$Url, [string]$Question) {
    $requestBody = @{
        question = $Question
        messages = @()
    } | ConvertTo-Json -Depth 6

    Invoke-RestMethod `
        -Method Post `
        -Uri $Url `
        -ContentType "application/json" `
        -Body $requestBody `
        -TimeoutSec 240
}

function Assert-ProjectAnswer($Response, [string]$PathName) {
    if ([string]::IsNullOrWhiteSpace($Response.answer)) {
        throw "$PathName returned an empty answer. Check the backend logs."
    }

    foreach ($project in @("EventSphere", "Social Welfare System", "ECA-IS")) {
        if ($Response.answer -notmatch [regex]::Escape($project)) {
            throw "$PathName answer did not include '$project'. Answer: $($Response.answer)"
        }
    }

    if ($Response.answer -notmatch "(?i)tech\s*stack") {
        throw "$PathName answer did not include the project tech stacks. Answer: $($Response.answer)"
    }

    if (-not $Response.sources -or $Response.sources.Count -lt 1) {
        throw "$PathName response did not include PDF source metadata."
    }
}

try {
    $health = Invoke-RestMethod -Uri "$BackendUrl/health" -TimeoutSec 10
    if ($health.status -ne "ok") {
        throw "Backend health endpoint returned status '$($health.status)'."
    }
    Write-Pass "FastAPI health endpoint is responding."

    $ollama = Invoke-RestMethod -Uri "$OllamaUrl/api/tags" -TimeoutSec 15
    $installedModels = @($ollama.models | ForEach-Object { $_.name })
    if ($installedModels -notcontains $ModelName) {
        throw "Ollama model '$ModelName' is missing. Installed models: $($installedModels -join ', ')"
    }
    Write-Pass "Ollama has model $ModelName installed."

    $question = "List all projects and the tech stack used in each project."

    $backendResponse = Invoke-YumiChat "$BackendUrl/api/chat" $question
    Assert-ProjectAnswer $backendResponse "FastAPI"
    Write-Pass "FastAPI RAG endpoint returned all three projects, stacks, and sources."

    $nextResponse = Invoke-YumiChat "$NextUrl/api/chat" $question
    Assert-ProjectAnswer $nextResponse "Next.js proxy"
    Write-Pass "Next.js API proxy returned all three projects, stacks, and sources."

    Write-Host "`nAll $script:passed YUMI RAG checks passed." -ForegroundColor Green
}
catch {
    Write-Host "`nYUMI RAG check failed: $($_.Exception.Message)" -ForegroundColor Red
    Write-Host "Check service status with: docker compose ps" -ForegroundColor Yellow
    Write-Host "Check logs with: docker compose logs --tail=100 backend ollama db" -ForegroundColor Yellow
    exit 1
}
```